



Trường Đại Học Công Nghệ Thông Tin, ĐHQG HCM

CS114.Q21

VIETNAMESE HATE SPEECH DETECTION - VIHSD

GVHD: TS. Võ Nguyễn Lê Duy

NHÓM 3

Thành viên:

23520587 Nguyễn Đức Hường
23520880 Nguyễn Bá Long
23521082 Nguyễn Thành Nhân
23521149 Phan Đức Thành Phát
23521304 Nguyễn Minh Quốc
23521329 Nguyễn Văn Quyền
23521413 Đỗ Hà Bình Thái

MỤC LỤC

I) Phát biểu bài toán

- 1) Giới thiệu đề tài
- 2) Ứng dụng
- 3) Bộ dữ liệu

II) Feature Engineering

- 1) Tạo & xử lý feature mới
- 2) Preprocessing
- 3) Feature selection

III) Method

- 1) Tổng quan pipeline
- 2) Feature Representation
- 3) Model training

IV) Web Demo

V) Q&A



I. PHÁT BIỂU BÀI TOÁN



1. GIỚI THIỆU ĐỀ TÀI

Bối cảnh thực tế:

- Sự bùng nổ của mạng xã hội (Facebook, TikTok, YouTube) tại Việt Nam dẫn đến sự gia tăng của ngôn từ độc hại.
- Ngôn từ thù ghét (Hate speech) đe dọa trực tiếp đến tâm lý người dùng và sự văn minh của môi trường mạng.

Thách thức đặc thù:

- Tiếng Việt có cấu trúc phức tạp, đa dạng về từ lóng, tiếng địa phương và từ ngữ châm biếm.
- Teencode và viết tắt (ví dụ: dm, vcl, clm...) khiến các bộ lọc từ khóa truyền thống trở nên vô hiệu.

Mục tiêu dự án:

- Xây dựng hệ thống tự động nhận diện ngôn từ thù ghét dựa trên bộ dữ liệu chuẩn UIT-ViHSD.
- Tối ưu hóa quy trình tiền xử lý và mô hình Học máy để đạt hiệu năng cao, phản hồi thời gian thực và thích ứng tốt với ngôn ngữ mạng hiện đại.

2. ỨNG DỤNG

Kiểm duyệt tự động cho mạng xã hội & Diễn đàn:

- Tích hợp trực tiếp vào mục bình luận của các nền tảng lớn (Facebook, TikTok) hoặc các trang báo điện tử tiếng Việt.
- Giảm tải từ 70% - 80% công việc và áp lực tâm lý cho đội ngũ nhân sự kiểm duyệt thủ công.

Quản lý danh tiếng thương hiệu cho Doanh nghiệp:

- Hỗ trợ các công ty tự động lọc các bình luận "troll", xúc phạm, hoặc phá hoại từ đối thủ trên Fanpage hệ thống, giữ hình ảnh thương hiệu sạch.

Ngăn chặn bạo lực mạng:

- Phát hiện sớm và ẩn các nội dung công kích cá nhân, bảo vệ người dùng (đặc biệt là học sinh, sinh viên) khỏi tổn thương tâm lý.

Hỗ trợ giám sát thông tin:

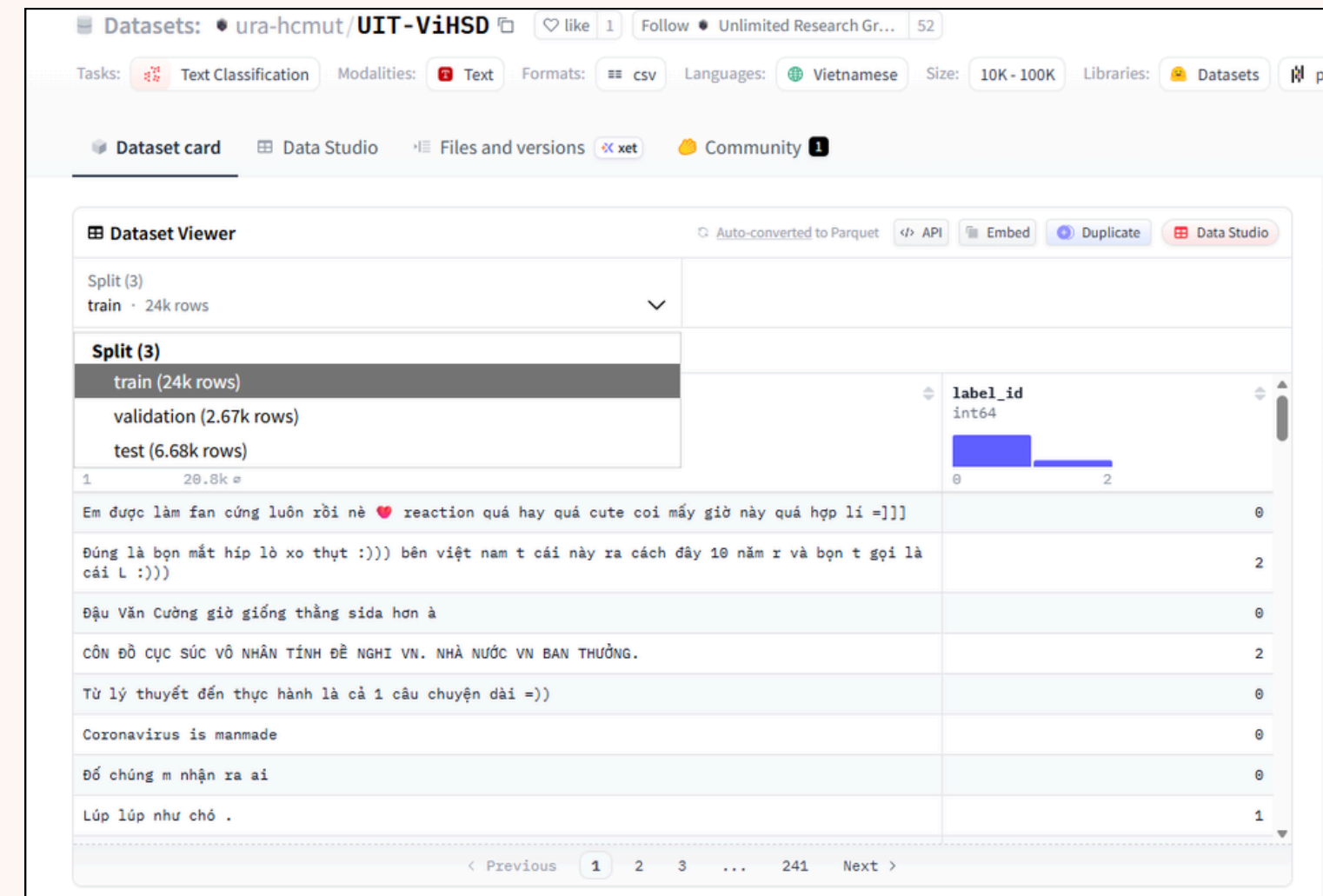
- Giúp các tổ chức theo dõi và đánh giá mức độ "toxic" (độc hại) của các cuộc thảo luận trực tuyến theo thời gian thực.



3. BỘ DỮ LIỆU

- Sử dụng dataset UIT-ViHSD được xây dựng bởi nhóm xử lý ngôn ngữ tự nhiên trường đại học công nghệ thông tin
- Nguồn dữ liệu Hugging face: <https://huggingface.co/datasets/ura-hcmut/UIT-ViHSD>

Bộ dữ liệu chứa khoảng 33000 câu bình luận lấy được thu thập từ các nền tảng mạng xã hội.



Datasets: ura-hcmut/UIT-ViHSD

Tasks: Text Classification Modalities: Text Formats: csv Languages: Vietnamese Size: 10K - 100K Libraries: Datasets

Dataset card Data Studio Files and versions xet Community

Dataset Viewer

Split (3)
train · 24k rows

Split (3)

label_id
0
2

1	20.8k
Em được làm fan cứng luôn rồi nè ❤️ reaction quá hay quá cute coi mấy giờ này quá hợp lí =]]]	0
Đúng là bọn mất híp lò xo thật :))) bên việt nam t cái này ra cách đây 10 năm r và bọn t gọi là cái L :)))	2
Đậu Văn Cường giờ giống thằng sida hơn à	0
CÔNG ĐỒ CỤC SÚC VÔ NHÂN TÍNH ĐỂ NGHI VN. NHÀ NƯỚC VN BAN THƯỜNG.	2
Từ lý thuyết đến thực hành là cả 1 câu chuyện dài =))	0
Coronavirus is manmade	0
Đồ chúng m nhận ra ai	0
Lúp lúp như chó .	1

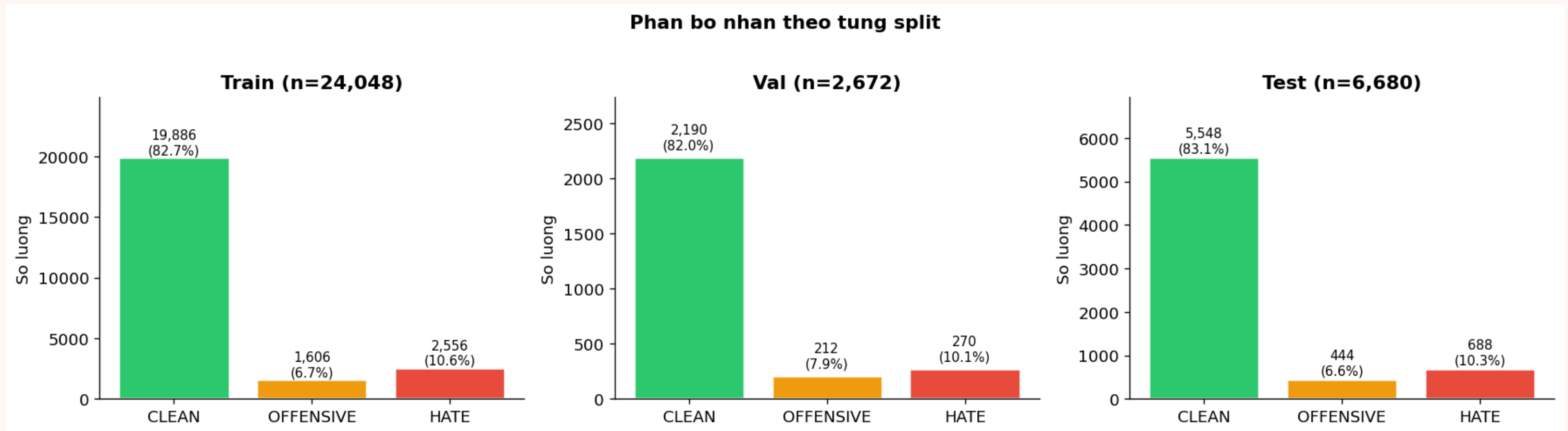
< Previous 1 2 3 ... 241 Next >



EXPLORATORY DATA ANALYSIS

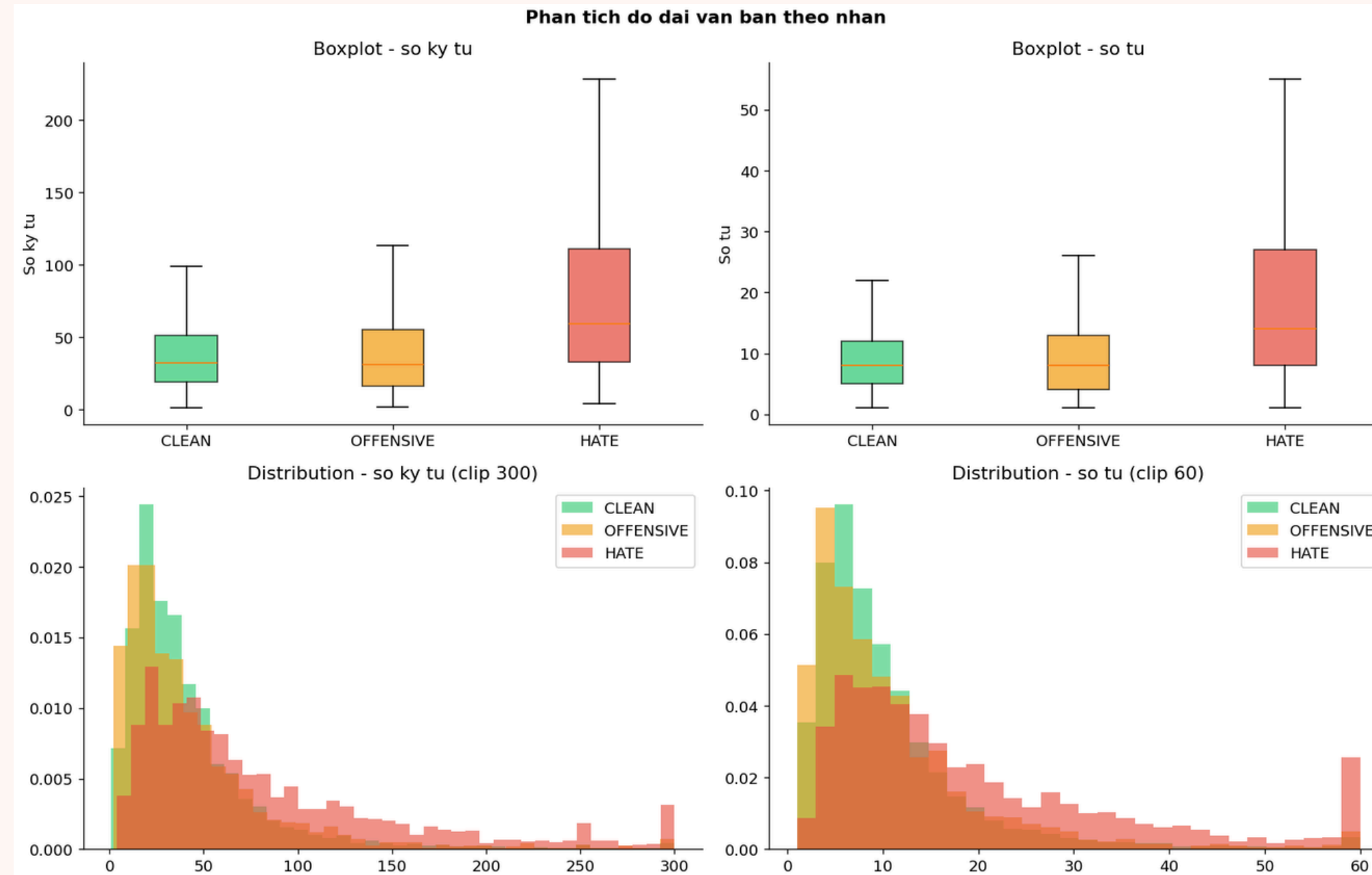


PHÂN BỐ NHÂN THEO TỪNG SPLIT



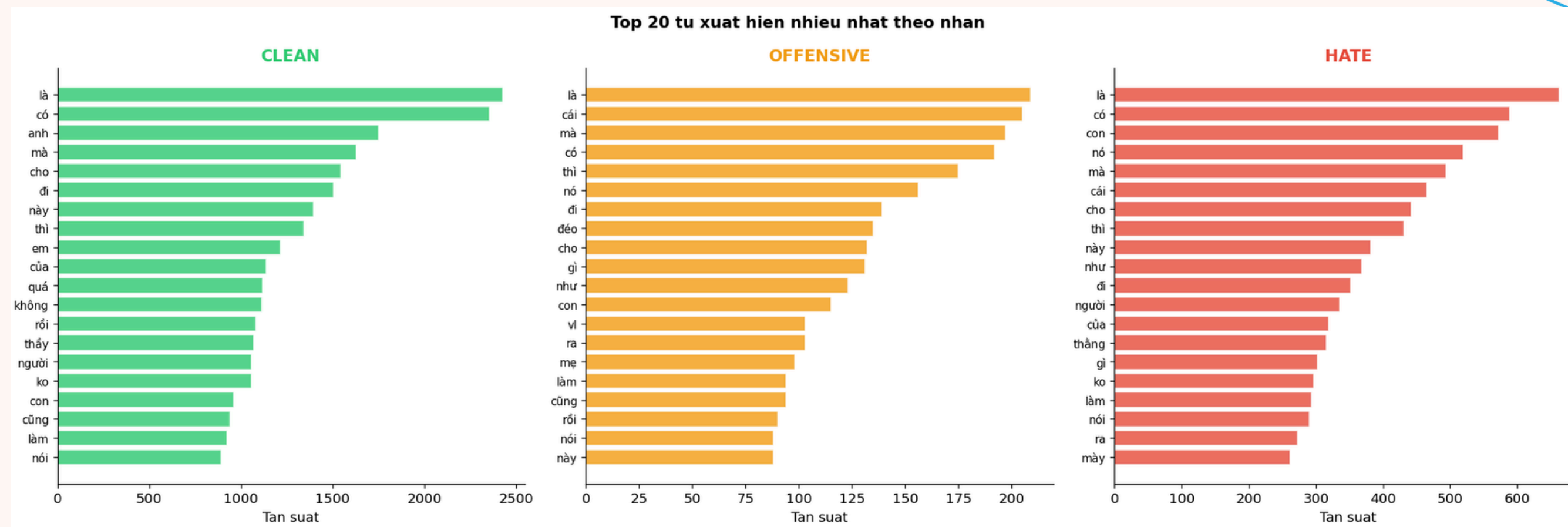
Dataset mất cân bằng nghiêm trọng

PHÂN TÍCH ĐỘ DÀI VĂN BẢN

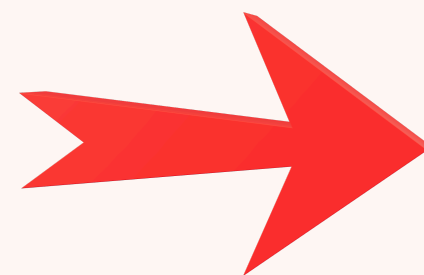


xác định xu hướng dài ngắn của câu bình luận => Phát hiện câu quá ngắn (có thể gây nhiễu) hoặc câu quá dài (mẫu hiếm gặp)

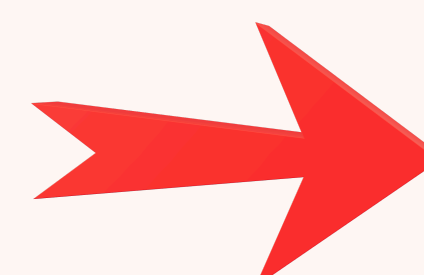
ĐẶC ĐIỂM TỪ VỰNG



token hóa đơn giản
bằng regex, lấy top-
20 từ theo nhãn, vẽ
wordcloud theo
nhãn

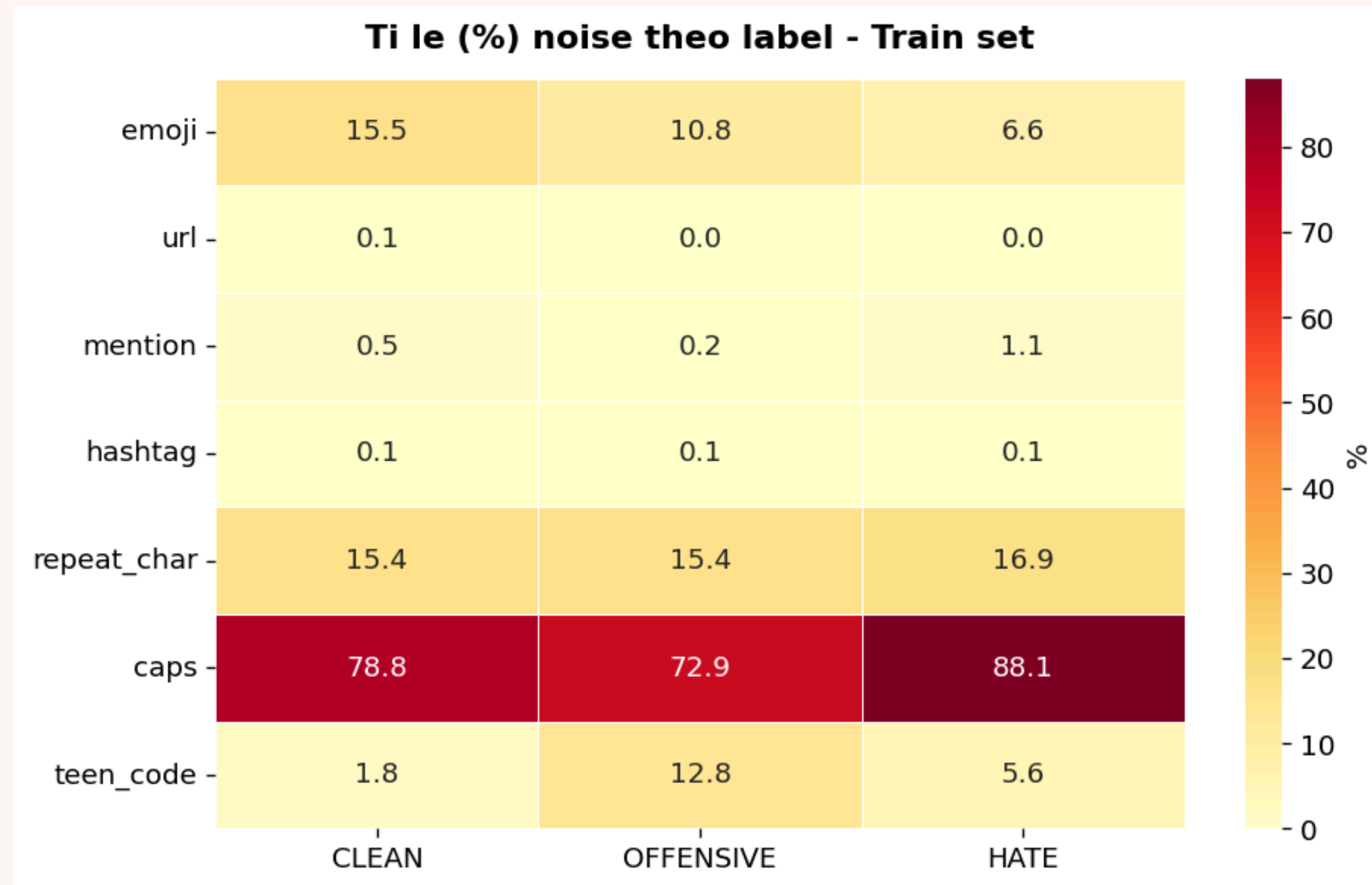


cho biết từ/cụm từ
nổi bật của từng lớp
và mức “phong phú”
từ vựng



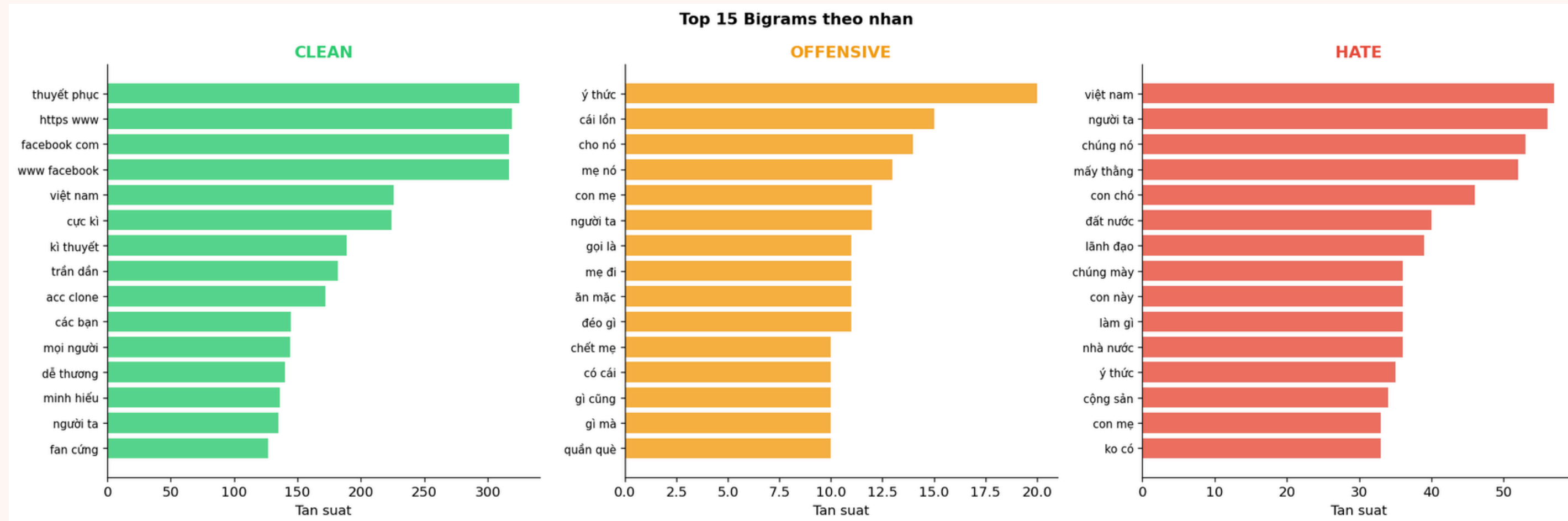
Các từ xuất hiện
nhiều nhất là các
stopword(‘là’, ‘và’, ‘thì’,
mà’) chứng tỏ dữ liệu
còn rất nhiều nhiễu

NOISE & ĐẶC THÙ TIẾNG VIỆT

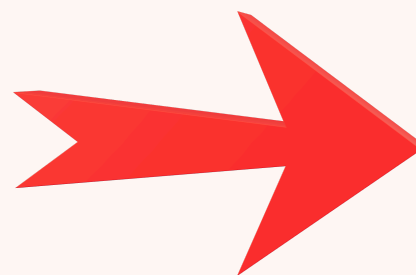


Thống kê một số dạng nhiễu thường xuất hiện trong các câu bình luận trên mạng xã hội như emoji, URL, hastag, caplock... theo từng nhãn

PHÂN TÍCH N-GRAM



Tìm hiểu xem các cụm từ (bigram) mang ý nghĩa phân loại như thế nào.



Nắm bắt được ngữ cảnh mà các từ đơn đôi khi không thể hiện

KIỂM TRA CHẤT LƯỢNG DỮ LIỆU

Rà soát dữ liệu khuyết thiếu: Phát hiện và xử lý các giá trị trống (Null values, Empty strings).

Lọc trùng lặp dữ liệu: Loại bỏ các mẫu bình luận giống nhau hoàn toàn (Exact duplicates).

Giải quyết mâu thuẫn nhãn: Sửa đổi các trường hợp cùng một câu nhưng bị gán hai nhãn khác nhau (Label conflicts).

Xử lý phần tử Outliers: Nhận diện các văn bản quá ngắn (chỉ có dấu) hoặc quá dài (số chữ bị hiếm gặp).



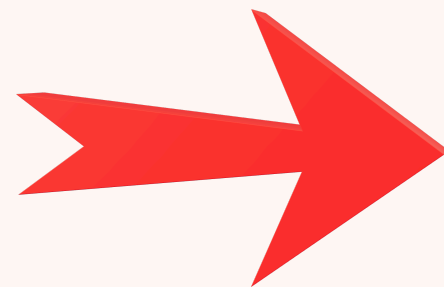
DATA PREPROCESSING



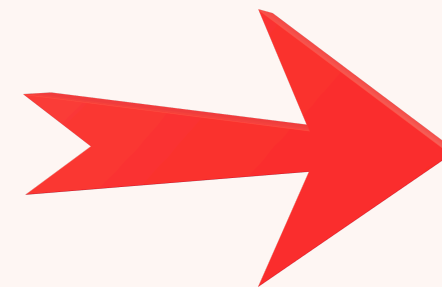


CHUẨN HÓA UNICODE

các bộ gõ tiếng việt khác nhau (TELEX, VNI...) có thể tạo ra các từ giống nhau nhưng cấu trúc mã HEX lại khác nhau



sử dụng chuẩn "NFC" của thư viện unicodedata để chuẩn hóa

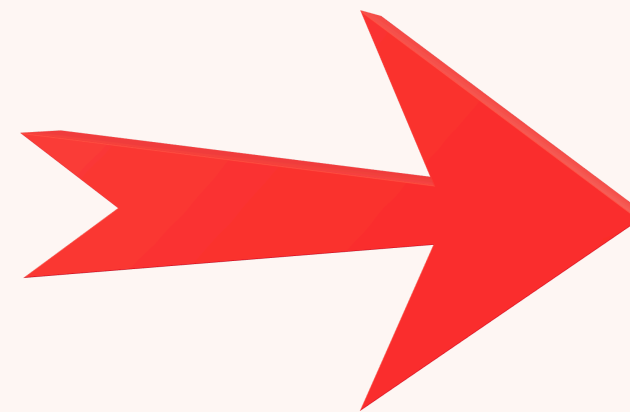


giúp thu gọn từ vựng tránh việc cùng một từ nhưng bị hiểu thành hai từ khác nhau



CHUẨN HÓA KỸ TỰ LẬP

đưa các từ bị kéo dài như “nguuuuu”, hay “đẹpppp” về đúng từ gốc



tránh việc mô hình không học đủ trọng số

LOẠI BỎ NHIỀU

Sử dụng REGEX (các biểu thức chính quy) để loại bỏ các link web (http/www...), Hastag (#tag) và toàn bộ emoji



loại bỏ các yếu tố gây nhiễu cho mô hình

xóa sạch các dấu câu như '!', '?', '.'



tránh trường hợp một cùng một nội dung bị hiểu thành hai do một bên có chứa dấu câu

PHIÊN DỊCH 'TEENCODE'

Các câu bình luận trên mạng xã hội thường sử dụng rất nhiều những từ ngữ thù ghét ở dạng viết tắt hoặc từ lóng.

Hướng giải quyết

định nghĩa một từ điển nhằm chuyển câu bình luận về dạng mà mô hình có thể hiểu được

ví dụ

các từ viết tắt như 'cc','vl','dm'... hay các từ lóng tiếng anh như 'wtf','lolol',...mô hình không thể trực tiếp hiểu các từ này



REMOVE STOPWORD

- loại bỏ các stopword như 'là', 'và', 'nhưng',...theo một từ điển tự định nghĩa
- Giữ lại các từ 'không', 'rất', 'quá' để không làm mất ý phủ định của câu. ví dụ câu '...không ngu...' nếu bỏ từ 'không' sẽ trở thành '...ngu...' đảo ngược hoàn toàn nhãn từ CLEAN thành OFFENSIVE





II. FEATURE ENGINEERING



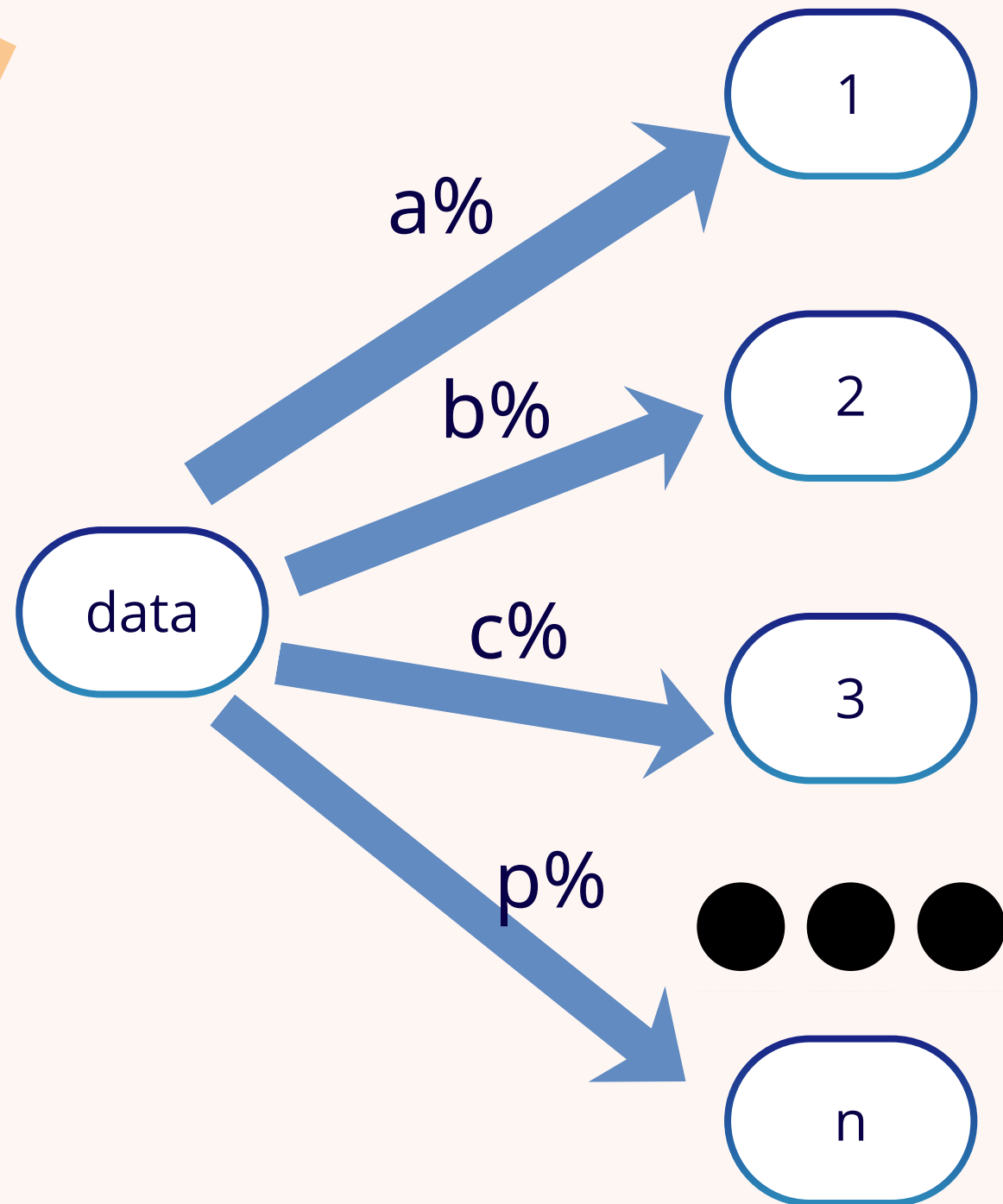
TF-IDF

Tần suất xuất hiện của từ (TF):
số lần xuất hiện của từ trong
một văn bản cụ thể

Tần suất nghịch đảo của tài
liệu (IDF): ước lượng độ quan
trọng của từ. từ có TF cao
nhưng xuất hiện ở nhiều tài
liệu thì có IDF thấp

Đây là xương sống của
pipeline. giúp mô hình hiểu
được từ nào xuất hiện nhiều
trong bình luận hiện nhưng
hiếm gặp ở các bình luận khác
→ mang tính đặc trưng cao

LDA

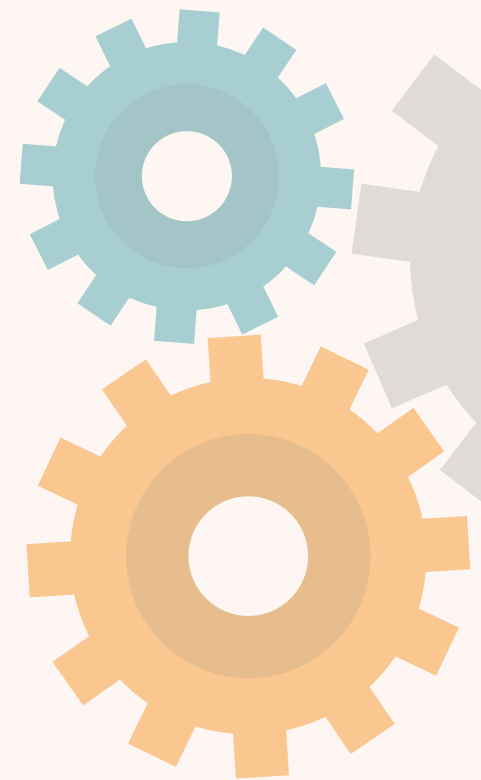


- gom cụm dữ liệu vào các chủ đề ẩn
- Thêm ngữ nghĩa cấp độ chủ đề, thay vì chỉ nhìn vào từng chữ mô hình → cung cấp thêm đặc trưng để mô hình phân loại câu bình luận



LEXICON

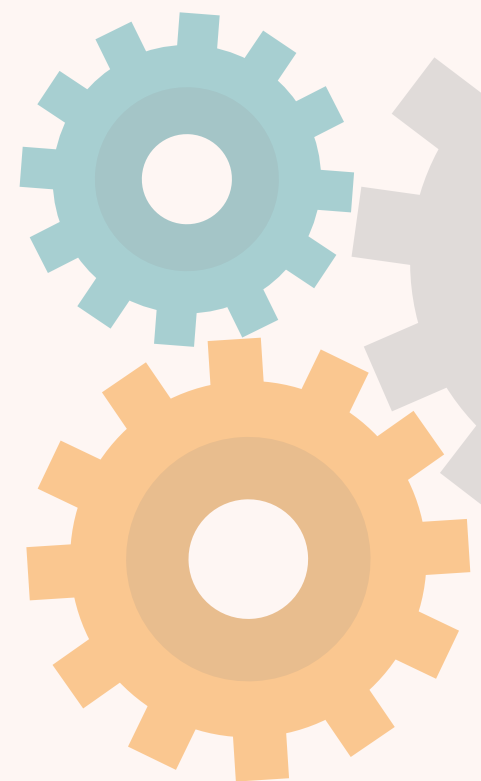
- Tự xây dựng một list các từ thù ghét + từ điển teencode được xây dựng ở bước tiền xử lý
- Kiểm tra số lượng từ chửi thề trong câu, tỉ lệ từ chửi thề trên tổng số từ, số lượng từ có độ dài hơn một n nhất định
- Đưa Domain Knowledge của con người vào mô hình





STATISTICAL FEATURE

- tính toán các thuộc tính bề mặt của câu: Độ dài câu, số từ, độ dài trung bình của từ...
- nắm bắt hành vi của người dùng





III. METHOD



1. Tổng quan pipeline

Tiền xử lý

Chuẩn hóa Unicode, loại bỏ noise (Emoji, Link), tách từ Tiếng Việt (Underthesea) và xử lý Teen code.

Trích xuất

Sử dụng TF-IDF Vectorizer với N-gram (1,2) để chuyển đổi văn bản sang không gian vector đặc trưng.

Huấn luyện

Thực hiện Cross-validation và Hyperparameter Tuning để tối ưu hóa chỉ số Macro F1-Score.



2.Feature Representation



FEATURE ĐẦU VÀO

Chiến lược: TF-IDF Vectorization

- đo mức độ quan trọng của từ
- giảm ảnh hưởng của các từ xuất hiện quá phổ biến.

Kết quả

- TF-IDF matrix shape: (23710, 30000)
- Sau feature selection: 15,000 features

$$w_{x,y} = tf_{x,y} \times \log \left(\frac{N}{df_x} \right)$$

TF-IDF

Term x within document y

$tf_{x,y}$ = frequency of x in y

df_x = number of documents containing x

N = total number of documents

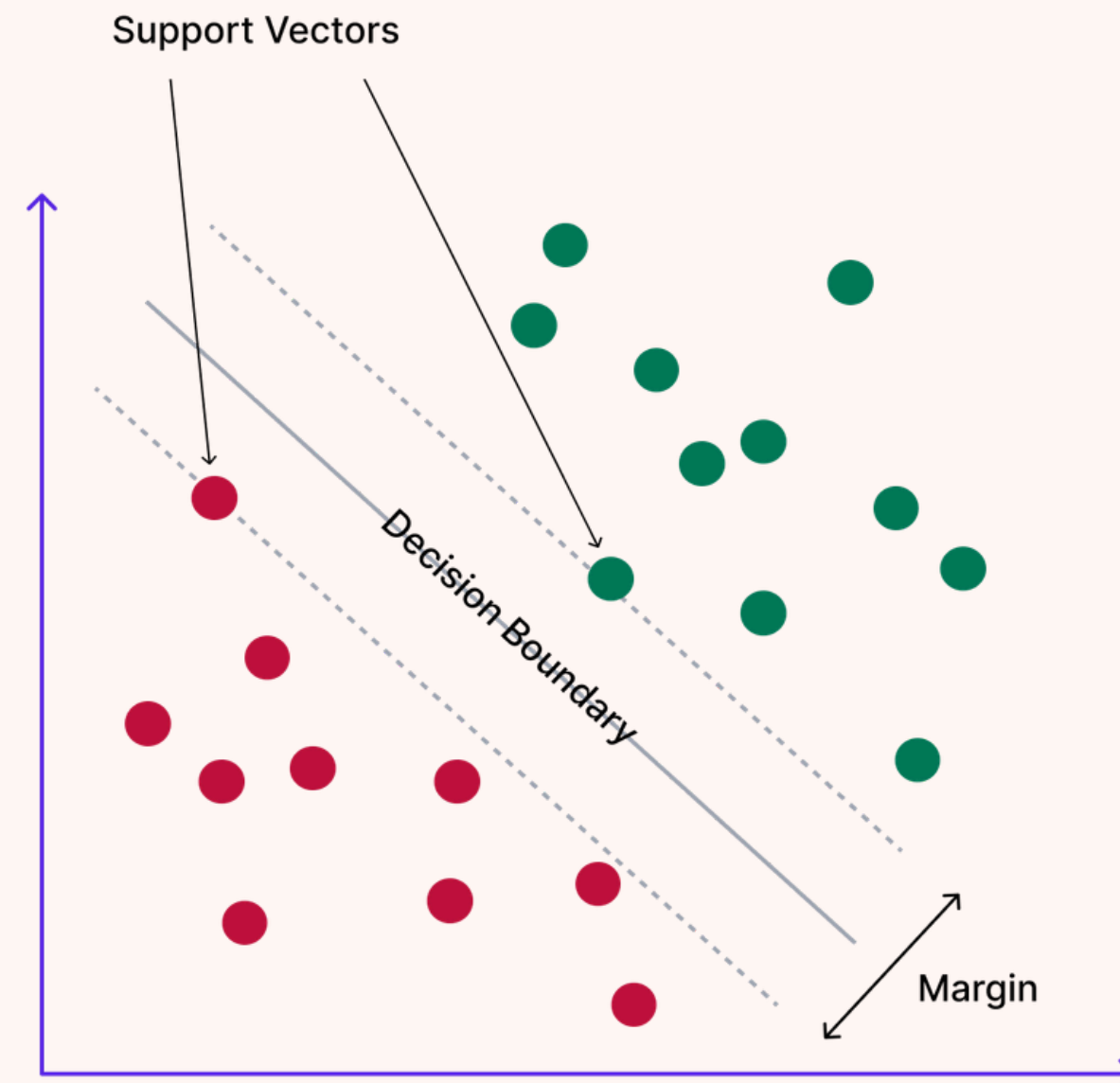
3. MODELS

LinearSVM Overview

- SVM tìm kiếm một siêu phẳng (hyperplane) có lề (margin) cực đại để phân tách các lớp dữ liệu trong không gian đa chiều.
- "Linear" vì dùng kernel tuyến tính — phù hợp với TF-IDF sparse high-dimensional features .

$$f(x) = \text{sign}(w^T x + b)$$

Anatomy of a Linear SVM



* Exponent

CONFIG & TRAINING

- **Cấu hình cốt lõi:** Sử dụng LinearSVC với tham số chính là $C = 0.5$ (Regularization parameter).
- **Tuning:** Thử nghiệm các ngưỡng xác suất (Thresholds) khác nhau để cân bằng giữa Hate và Offensive labels.

- **`class_weight='balanced'`**

Dataset bị mất cân bằng:

CLEAN chiếm đa số

HATE/OFFENSIVE chiếm thiểu số

Balanced weighting giúp:

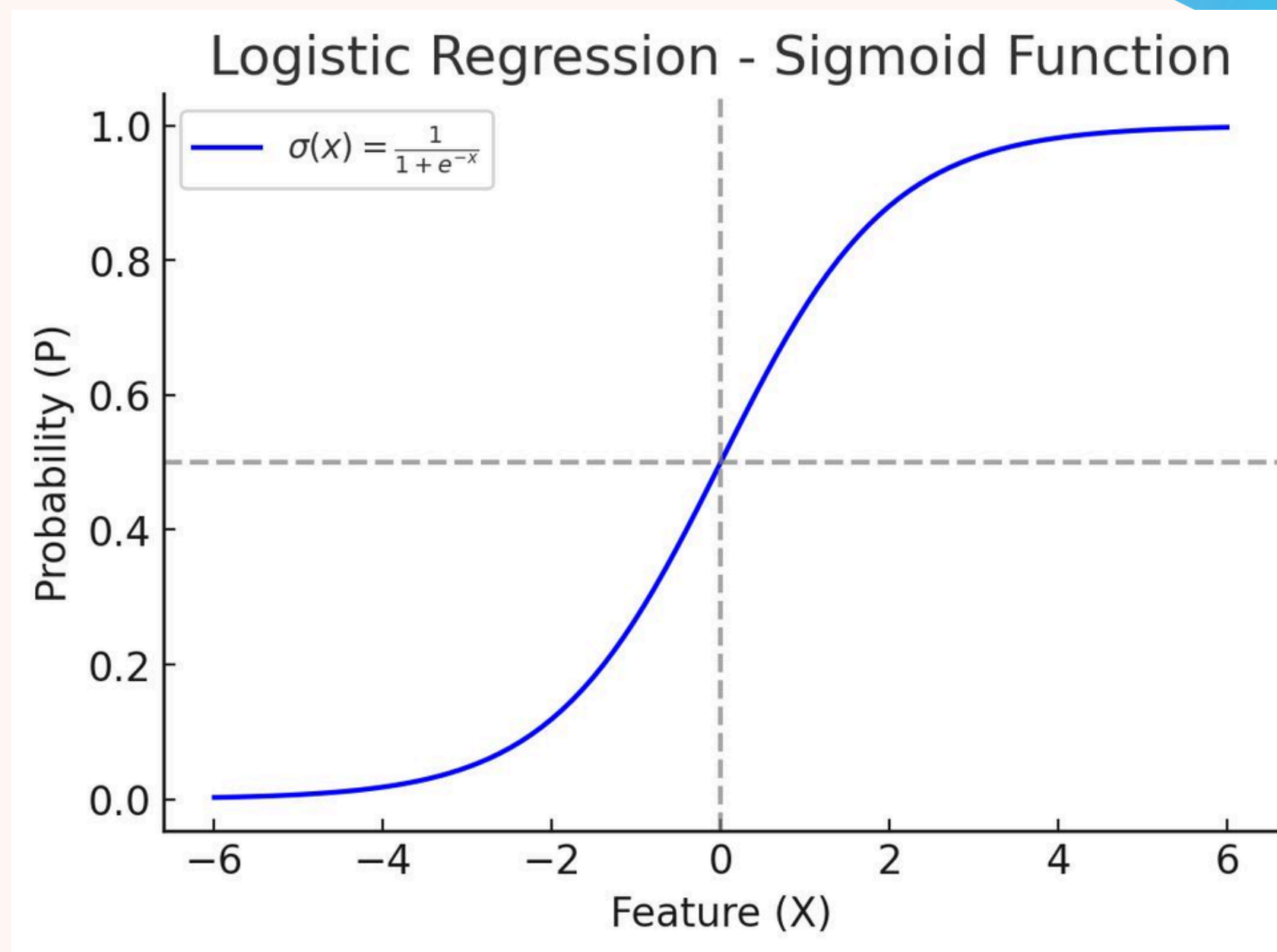
tăng attention cho minority classes.

- **Training time:** Hiệu suất huấn luyện ấn tượng (~70s), phù hợp cho việc triển khai thực tế.

3. MODELS

Logistic Regression Overview

- Mô hình tuyến tính dùng hàm Sigmoid/Softmax để cho xác suất từng lớp
- Phân loại đa lớp qua Multinomial (One-vs-Rest)
- Khác LinearSVM ở chỗ: output là xác suất → dễ áp dụng Threshold Tuning hơn
- Giải thích được bằng LIME (feature importance)



CONFIG & TRAINING

- **Xử lý đa lớp:** Sử dụng chiến lược multinomial (Softmax) để phân loại đồng thời 3 nhãn.
- **Tham số:** Tối ưu hóa $\text{penalty}='l2'$ và $\text{solver}='lbfgs'$ để đảm bảo độ ổn định toán học.

- **L2 Regularization**

Giúp:

giảm overfitting

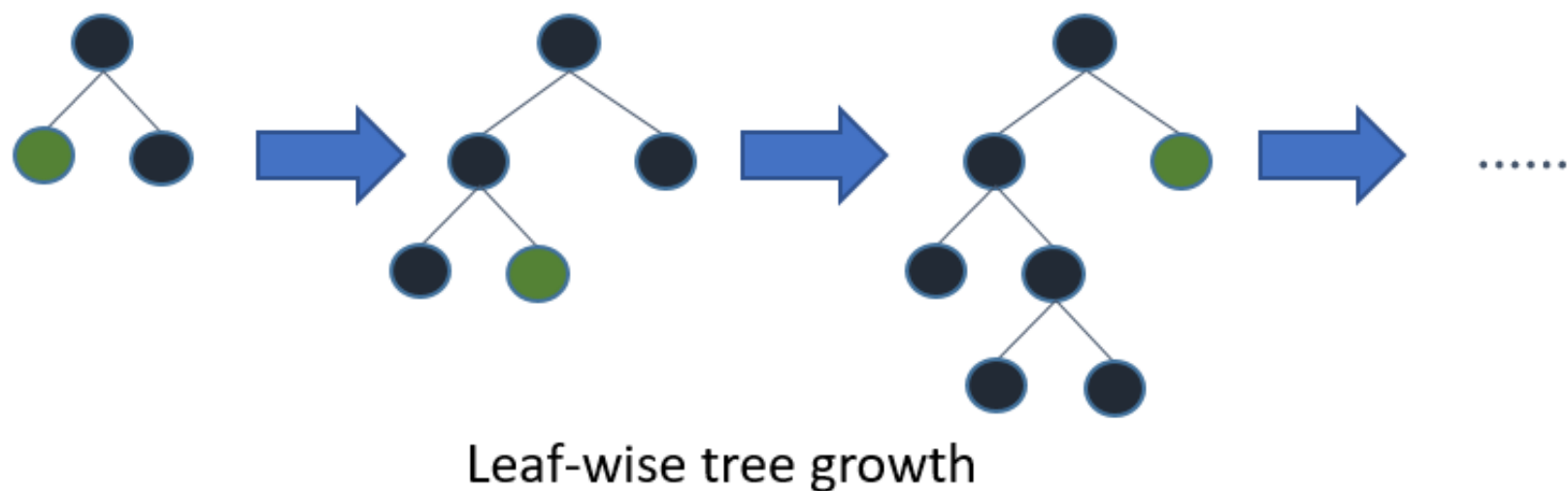
tránh weight quá lớn

- **Training time:** Dài hơn SVM (~496s) do tính toán xác suất phức tạp trên dữ liệu lớn.

3. MODELS

LightGBM Overview

- Gradient Boosting: xây chuỗi cây quyết định, mỗi cây sửa lỗi của cây trước
- Leaf-wise growth (khác XGBoost dùng Level-wise) → hội tụ nhanh hơn
- Histogram-based splitting → xử lý nhanh dù feature nhiều



$$\mathcal{L}^{(t)} = \sum_{i=1}^n l\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t)$$

Trong đó:

- l là hàm mất mát (ví dụ: MSE, Logloss).
- y_i là giá trị thực tế của mẫu thứ i .
- $\hat{y}_i^{(t-1)}$ là giá trị dự đoán của mẫu i qua $t - 1$ cây trước đó.
- $f_t(x_i)$ là cây quyết định đang được xây dựng ở bước t .
- $\Omega(f_t)$ là số hạng điều chuẩn (regularization) nhằm tránh hiện tượng quá khớp (overfitting). NB Michael Brenndoerfer +3

CONFIG & TRAINING

- **Chiến lược:** Sử dụng cây phân loại rời rạc để học các đặc trưng phi tuyến tính phức tạp từ văn bản.
- **Cấu hình:** Tập trung vào num_leaves, learning_rate và feature_fraction.

n_estimators=200, learning_rate=0.1, num_leaves=31, class_weight='balanced'

Leaf-wise Tree Growth

LightGBM:

- phát triển tree theo leaf-wise
 - giúp giảm loss nhanh hơn level-wise.
- **Training time:** Tốc độ xử lý song song vượt trội, chỉ mất ~139s cho một cấu trúc Boosting phức tạp.



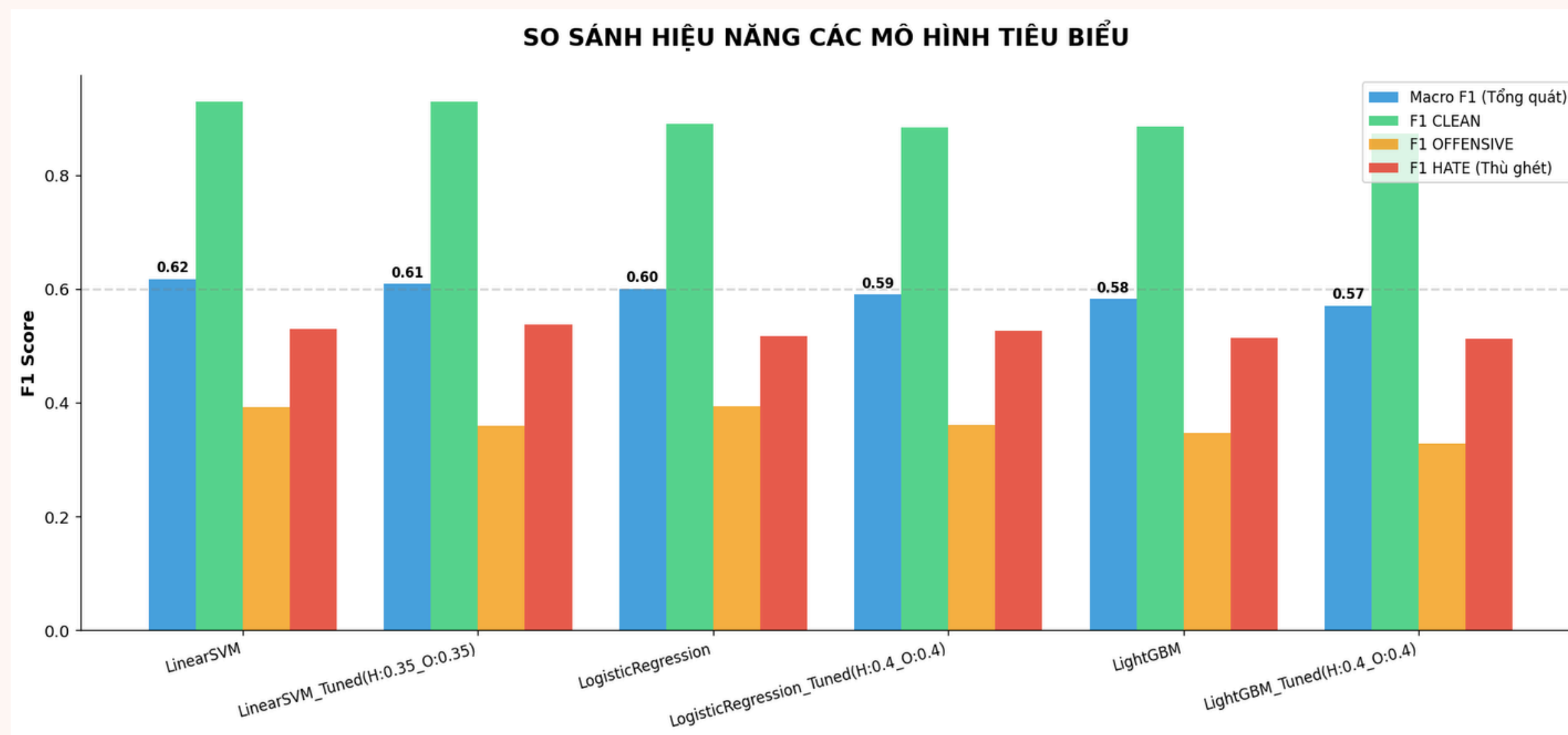
KẾT QUẢ THỰC NGHIỆM



BẢNG SO SÁNH HIỆU NĂNG

• KẾT QUẢ TRÊN TẬP TEST:

Mô hình (Model)	Accuracy(Cao hơn là tốt)	Macro F1 (Cao hơn là tốt)	Train Time
LinearSVM	0.8580	0.6172	70.83
Logistic Regression	0.7991	0.6005	496.09
LightGBM	0.7933	0.5823	139.04



• PHÂN TÍCH :

- LinearSVM đạt hiệu suất tốt nhất với Macro-F1 cao nhất (~0.62), cho thấy các mô hình tuyến tính hoạt động hiệu quả trên đặc trưng TF-IDF sparse vectors.
- OFFENSIVE là class khó phân loại nhất do semantic ambiguity và mất cân bằng dữ liệu, trong khi threshold tuning chỉ cải thiện nhẹ hiệu suất minority classes.
- Chỉ số Macro F1 được ưu tiên vì tính chất mất cân bằng nhãn của tập dữ liệu ViHSD, đảm bảo mô hình nhận diện tốt cả Hate và Offensive thay vì chỉ tập trung vào Clean.



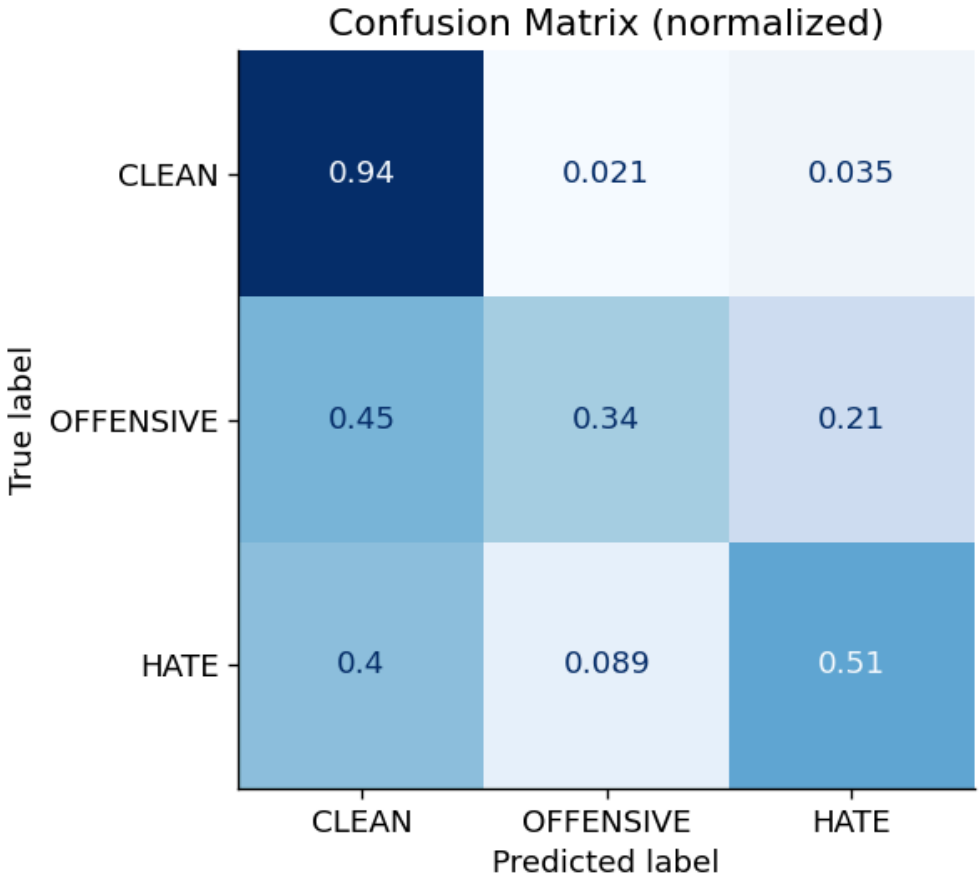
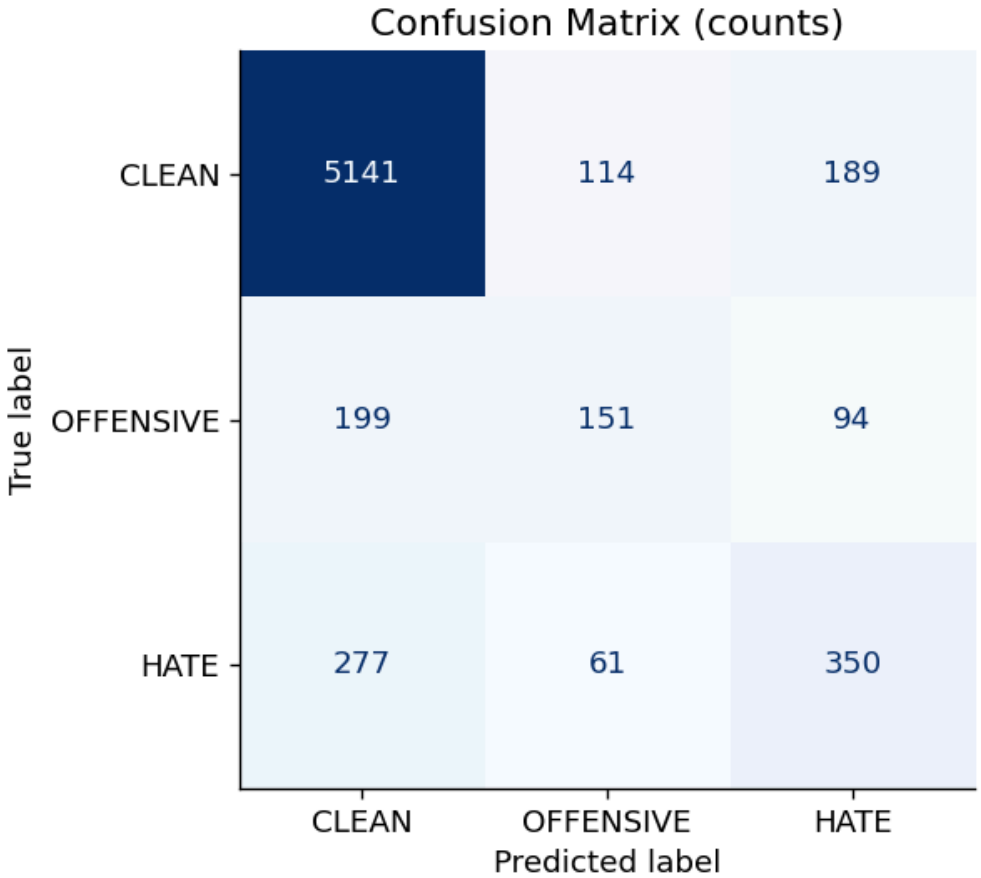
Best model: LinearSVM

Macro F1 = 0.6172

Accuracy = 0.8580

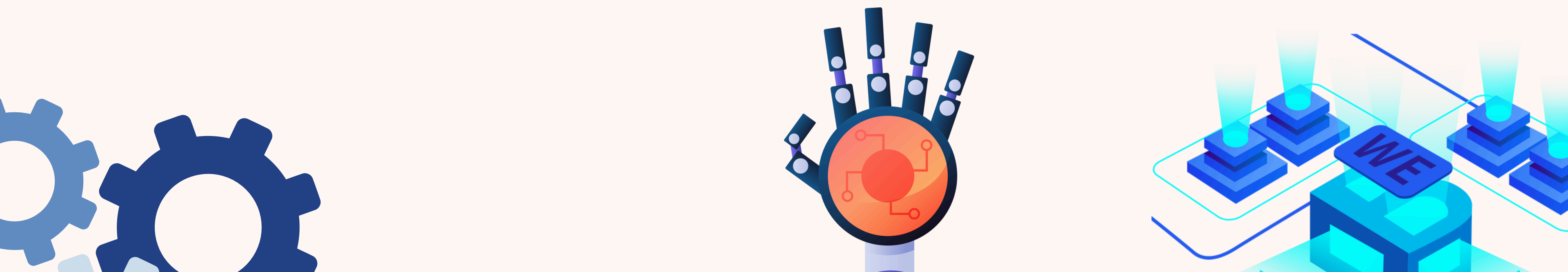
	precision	recall	f1-score	support
CLEAN	0.92	0.94	0.93	5444
OFFENSIVE	0.46	0.34	0.39	444
HATE	0.55	0.51	0.53	688
accuracy			0.86	6576
macro avg	0.64	0.60	0.62	6576
weighted avg	0.85	0.86	0.85	6576

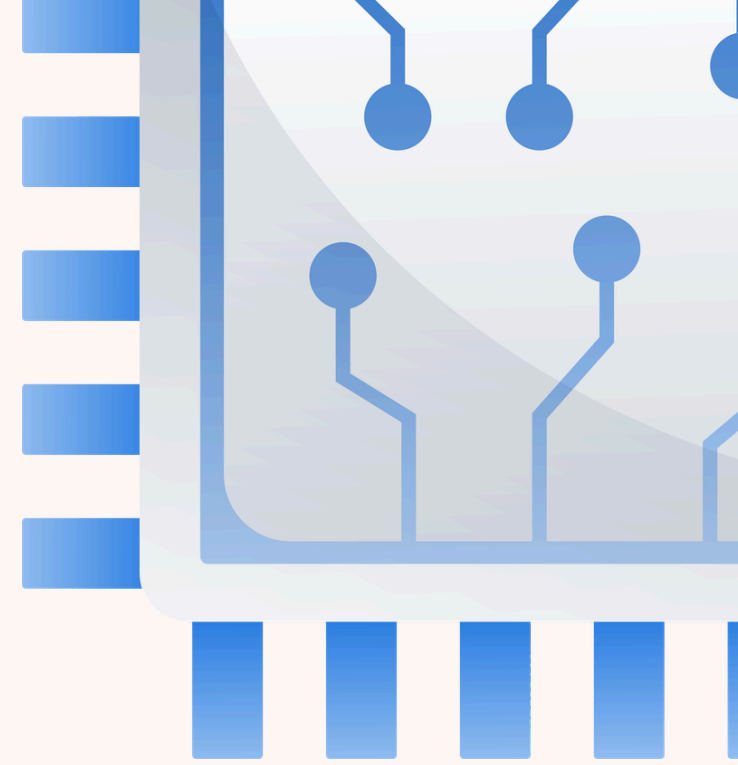
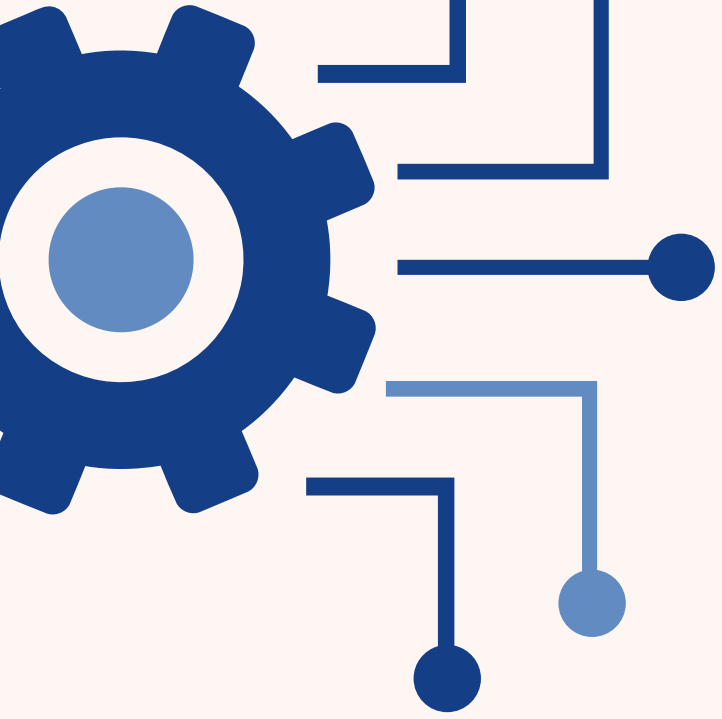
Evaluation - Best model: LinearSVM





IV. WEB DEMO





**THANK
YOU**

